

Proyecto 01: Gestión de un Hospital.

Este documento contiene las especificaciones de funcionalidad y requerimientos para el primer proyecto de programación de la asignatura **Estructura de Datos y de la Información** en el curso 2022/23.

1. Introducción.

Nuestro objetivo será desarrollar una aplicación que realice la gestión básica de personal y consultas en un hospital. La aplicación tendrá una funcionalidad muy limitada, pero podrá ser extendida posteriormente. Por tanto, debe cumplir con las siguientes características generales:

- **Eficiencia:** debido a la gran cantidad de datos y al esperado crecimiento de los mismos en el futuro, debemos encontrar la forma de almacenar y procesar los datos de la forma más eficiente posible. Además, deberemos realizar un análisis de la complejidad de nuestra aplicación para cumplir este requisito.
- **Extensibilidad:** el diseño de la aplicación en cuanto a entidades y estructuras de datos debe ser extensible, para facilitar el desarrollo de futuras versiones de la aplicación y ampliar su funcionalidad.
- **Modularidad:** la organización de las entidades que forman nuestra aplicación debe ser modular, de forma que las modificaciones y ampliaciones de la misma se puedan realizar de la forma más sencilla posible, afectando a la menor cantidad de código.

Para cumplir con los requisitos anteriores se seguirá una metodología de desarrollo basada en el paradigma de *Programación Orientada a Objetos* y se utilizará el lenguaje de programación **C++**.

Este documento se organiza de la siguiente forma. En la **sección 2** se especifica el código a partir del cual se puede comenzar la implementación del proyecto. En la **sección 3** se enumeran los algoritmos básicos que debemos implementar y en la **sección 4** se discuten los mecanismos de análisis y tests que debemos aplicar sobre nuestra aplicación para evaluarla, y asegurar que cumple los requisitos y lo hace de forma eficiente. Finalmente, la **sección 5 establece los criterios de evaluación** de la calidad del desarrollo y su calificación, así como las fechas de entrega.

2. Creación del código base para el proyecto.

El proyecto contendrá las clases implementadas durante las sesiones de laboratorio (algunas proporcionadas por el profesorado y disponibles en el aula virtual):

1. **Paciente, Medico, Consulta y FechaYHora.**
2. **VoV_Pacientes, VoV_Medicos y VoV_Consultas.** Recordemos que estas clases implementan un vector de ocupación variable y que, en el caso de las **consultas**, **debe estar ordenado por fecha y hora.**
3. La clase **Hospital**. Es la clase principal del proyecto.
4. Los archivos **.csv** con los datos que hay que importar (**pacientes.csv**, **medicos.csv** y **consultas.csv**).
5. Un programa principal que implementa un *menú* para ejecutar diferentes algoritmos.

El esqueleto de código para este menú lo podéis descargar del Aula Virtual.

A partir de la estructura anterior, se incorporará nueva funcionalidad al Hospital en forma de algoritmos que se ejecutarán a partir del menú.

Implementación de las clases base.

Las clases `Paciente`, `Medico` y `Consulta` representan los objetos de nuestra aplicación. La elección de los atributos de cada clase y su representación (tipos de datos) es responsabilidad del programador, y se deben ajustar a los datos incluidos en los archivos y a lo establecido en las prácticas de la asignatura.

En concreto, los atributos *medico* y *paciente* de los objetos de tipo consulta serán **punteros a objetos** de las clases `Medico` y `Paciente`.

Las operaciones a implementar, establecidas en las sesiones de laboratorio, serán en general para cada clase: (1) *setters/getters* para cada atributo, (2) constructores, (3) destructor, (4) operadores sobrecargados necesarios, y (5) una operación para mostrar la información en pantalla. A estas operaciones básicas se podrán añadir otras si se considera necesario.

Datos.

Los datos a importar han sido pre-tratados y están a disposición del programador en el Aula Virtual de la asignatura en forma de ficheros `.csv`. Se deben utilizar los datos proporcionados en los ficheros **sin modificaciones**. Cualquier modificación supondrá que los resultados de los algoritmos no serán los esperados y, como consecuencia, podrán considerarse incorrectos.

Los archivos que contienen los datos son de tipo texto y se enumeran en la tabla siguiente:

<code>pacientes.csv</code>	Fichero que contiene los pacientes del hospital. La información de cada paciente será: <i>DNI, Nombre, apellidos, género</i> (0: Hombre, 1: Mujer, 2: NoDefinido) y <i>edad</i> .
<code>medicos.csv</code>	Fichero que contiene los médicos del hospital. La información de cada médico será: <i>Nombre, apellidos</i> y <i>especialidad</i> .
<code>consultas.csv</code>	Fichero que contiene las consultas del hospital. La información de cada consulta será: <i>DNI</i> del paciente, <i>Apellido</i> del médico, <i>Tipo de consulta</i> (0: Pendiente, 1: Urgente, 2: Externa), y <i>fecha y hora</i> .

Cada paciente, médico o consulta aparece en una línea del fichero correspondiente, y los campos se separan por el carácter `‘;’`.

Hay que tener en cuenta que algunas de las consultas en el fichero `consultas.csv` no contienen una fecha y una hora, puesto que no están programadas. En estos casos, aparecerá un carácter `‘-’`. Es labor del programador importar estos datos a la aplicación de forma correcta.

Contenedores de datos.

La estructura de datos básica que vamos a utilizar en el proyecto es el **Vector de Ocupación Variable**. La implementación de dicha estructura corresponde al alumno conforme a lo

establecido en clases de teoría y laboratorio. En el caso del almacenamiento de las **consultas**, los datos estarán **ordenados**. En el caso de **Pacientes y Médicos** no es necesario ordenarlos, aunque se deja a elección del alumno su ordenación y el criterio para la misma.

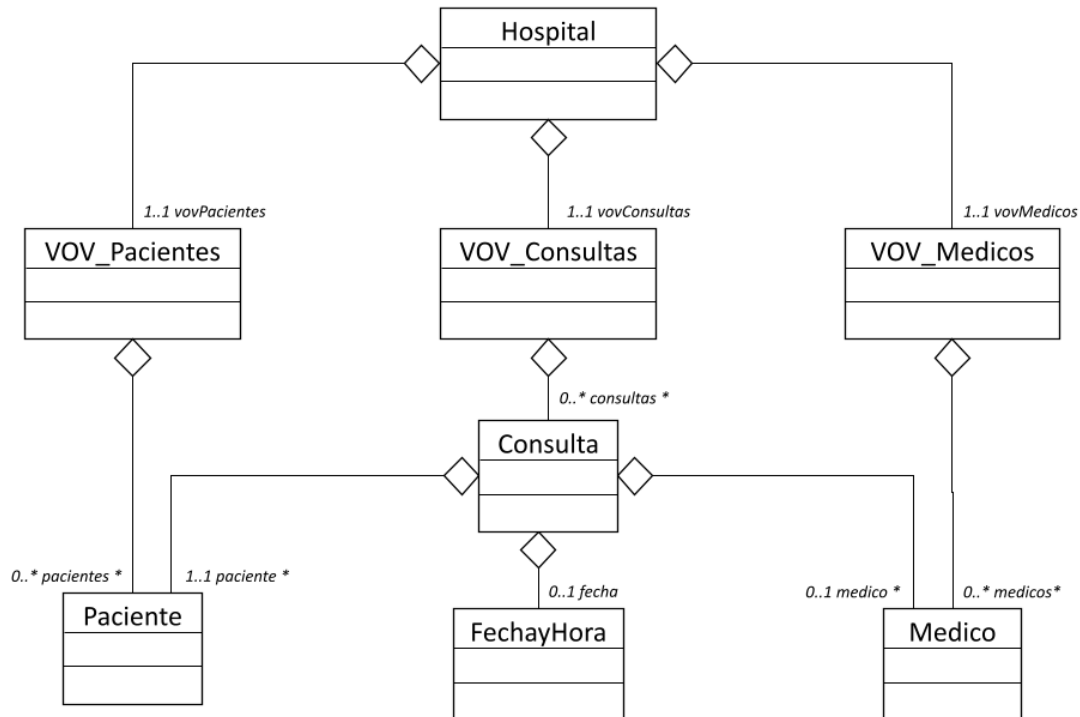
La interfaz del vector de ocupación variable para cada tipo de datos (pacientes, médicos y consultas) lo formarán las siguientes operaciones:

- Constructor por defecto. El vector tendrá una longitud especificada como una constante en la clase. El vector contendrá **punteros a objetos**.
- Destructor.
- Insertar un elemento (**puntero a objeto**) en el vector. Lo hará en la primera posición libre (`insertar`). No se insertará un elemento si el vector está lleno. En el caso de consultas, la inserción se hará en orden.
- Borrar un elemento por posición. Si la posición a borrar es mayor que el número de elementos del vector, la operación no realizará ninguna acción. En el caso de las consultas, el borrado debe respetar el orden.
- Consultar elemento por posición. Se devuelve el elemento en la posición especificada como parámetro. Si la posición es mayor que el número de elementos, la operación no realiza ninguna acción (`getPosicion`).
- Devolver el número de elementos en el vector (`getOcupadas`).
- Operación que devuelve `true` si el vector está vacío (`estaVacio`).
- Operación que devuelve `true` si el vector está lleno (`estaLleno`).

De forma **opcional**, se permite la implementación de un vector de ocupación variable mediante **templates de C++**, siguiendo la interfaz definida anteriormente. Este código evita duplicidades y lleva a una implementación más flexible, extensible y mantenible. Tened en cuenta que la interfaz del vector de ocupación variable debe ser correcta, y no debe incluir (por ejemplo) dos métodos para insertar e insertar en orden.

Relaciones entre clases.

El siguiente diagrama *UML* muestra la estructura de clases del proyecto:



3. Algoritmos a implementar.

Los algoritmos a implementar en nuestra aplicación son los siguientes:

1. Mostrar estadísticas del hospital: número de pacientes, número de médicos y número de consultas.
2. Mostrar todos los Pacientes.
3. Mostrar todos los Médicos.
4. Mostrar todas las Consultas (aparecerán en orden, puesto que el vector está ordenado).
5. Buscar un paciente por DNI. El método debe **devolver** (sin mostrar) el paciente que tiene el DNI solicitado. El paciente obtenido al llamar a este método (si se localiza) se mostrará fuera del método *buscar*.
6. Buscar un médico por apellido. El método debe devolver (sin mostrar) el médico que tiene el apellido solicitado. El médico obtenido, como antes, se mostrará (si se localiza) fuera del método *buscar*.
7. Guardar en un fichero (cuyo nombre será el DNI del paciente y la extensión .txt) todas las consultas que tiene *programadas* ese paciente, incluyendo, para cada consulta, exclusivamente el nombre y apellidos del doctor, su especialidad y la fecha y hora de la consulta.

Todos los algoritmos deben ser implementados sobre las estructuras de datos de tipo vector en las que, previamente, se habrá cargado la información desde fichero, es decir, los algoritmos anteriores (por ejemplo, búsqueda de un paciente o un médico) no podrán implementarse directamente sobre los datos del fichero.

4. Análisis, pruebas y documentación.

Además del código que implementa la funcionalidad descrita en las secciones anteriores, para obtener la calificación de aprobado será obligatorio incluir:

- Juego de **pruebas** unitarias **para cada clase implementada**, incluyendo su diseño e implementación en las librerías `pruebaClase.h` y `pruebaClase.cpp`, correspondientes. Se podrá realizar una prueba conjunta de los constructores y los *setters/getters* en las clases base (`Paciente`, `Medico` y `Consulta`).
- La documentación interna correspondiente a cada operación de las clases implementadas. Esta documentación incluirá *precondiciones*, *descripción* y *complejidad*. Se documentarán también los aspectos que el alumno considere oportunos sobre los detalles de su código.

5. Evaluación.

La rúbrica con la que se evaluará el proyecto está disponible en la actividad de entrega. Tal y como se indica en ella, los requisitos mínimos que deben cumplir los proyectos entregados para ser evaluados, son los siguientes:

- Tal y como se indica en la ficha de la asignatura, *la copia o el plagio en cualquier actividad, proyecto o prueba, ya sea en una parte o en su totalidad, supone una nota final de SUSPENSO (0) en la convocatoria y una nota de 0 en todas las calificaciones obtenidas hasta el momento para todas las personas implicadas, además de las actuaciones legales pertinentes.*
- Aquellos proyectos que no compilen o que no funcionen correctamente en la mayoría de los casos, no se corregirán y serán calificados con una nota de 0.
- Del mismo modo, aquellos proyectos que no incluyan la documentación interna de todas las clases o pruebas significativas para todas ellas, tampoco serán corregidos y serán calificados con una nota de 0.
- Aquellos proyectos en cuyas estructuras de datos se almacenen objetos en lugar de punteros a objetos obtendrán una calificación con una nota de 0.
- Para la implementación del proyecto solo podrá utilizarse C++. No está permitido utilizar la biblioteca STL (*Standard Template Library*) en el desarrollo, ni cualquier otra biblioteca similar.

Los proyectos entregados deberán cumplir las siguientes normas:

- Deberán realizarse **de forma individual**.
- Deberán entregarse en un fichero `zip` cuyo nombre seguirá el formato `ApellidosAlumno.zip` y cuyo contenido incluirá los archivos fuente (`.h` y `.cpp`) del código y pruebas desarrolladas, con su documentación interna.

Aquellos proyectos que cumplan los requisitos y normas anteriores, se evaluarán teniendo en cuenta los siguientes criterios:

- Uso de **punteros** en estructuras de datos, evitando así los problemas derivados de mover datos en memoria (*eficiencia*) y mantener distintas copias (*consistencia*).
- Diseño correcto de interfaces, incluidas la documentación (*precondiciones* y *descripción*) y evaluación de la *complejidad* de las operaciones.
- Organización correcta de los datos en memoria de forma que la implementación de los algoritmos cumpla con los requisitos de eficiencia y uso de memoria.
- Corrección en el resultado de la ejecución de los algoritmos.
- Uso correcto y eficiente de parámetros en la invocación de operaciones.
- Uso de constructores y destructores para las clases de forma coherente y adecuada, incluido el constructor por copia.
- Siempre que sea posible, no se duplicarán objetos en la aplicación, y se evitarán copias y duplicados de los mismos.

La fecha límite de entrega del proyecto es el 10 de marzo a las 23:55 h, a través de la correspondiente tarea abierta en el Aula Virtual de la asignatura. No se tendrán en cuenta entregas realizadas por cualquier otro medio que no sea el establecido, ni con posterioridad a esa fecha.